

SkyBook

Airline Reservation Platform

Platform Evolution & Engineering Toolchain

A case-study addendum

Praveenreddy Somireddy

Author & Engineer

7 August 2026

About this document

This addendum records how the SkyBook platform advanced beyond the version documented in the main case study, and — the part it is written to make legible — the **real-world engineering toolchain** the platform is built, tested, shipped and operated with. Every figure here is measured from the repository at the time of writing, not estimated.

SkyBook is a microservice airline reservation system: **one API gateway and seven domain services** (auth, flight, booking, inventory, payment, notification, check-in) behind a React single-page front end, communicating over HTTP at the edge and Apache Kafka between services. It runs on a single free-tier cloud VM behind a TLS front door, promoted there by a full CI/CD pipeline.

Author: **Praveenreddy Somireddy**. The design decisions, the trade-offs, and the defects found and fixed in production described below are the author's own work on the platform.

Part 1 — Platform evolution

Six capabilities were added or reworked. Each is a pattern real airlines run; each is described with what it does, why it is realistic, and the engineering detail that made it non-trivial — including, in three cases, a defect that only surfaced in production and how it was diagnosed and closed.

Sign in with Google (OIDC)

What & why. Passengers sign in — and implicitly register — with a Google account, the “Continue with Google” pattern every consumer travel site offers. Live in production behind the platform’s TLS domain.

Engineering. Authorization-code flow with PKCE. The governing rule is **Google authenticates; SkyBook authorizes**: the external identity is exchanged inside auth-service for the platform’s own RS256 token, so the gateway and all seven services never learn that SSO exists. Two production-only bugs were found and fixed by live use — a sealed OAuth state cookie that grew past both the browser’s 4 KB cap and nginx’s proxy-buffer limit (invisible to in-JVM tests), and relative redirects that resolved to an internal hostname behind the proxy chain. Both now have regression tests pinning the exact failure.

Guest check-in (no account required)

What & why. A travel agency books a ticket and sends the passenger an e-ticket; the passenger, who has no SkyBook account, retrieves the booking by reference and surname, checks in, downloads a boarding pass, and emails it to any address. This is the standard “Manage My Booking” model.

Engineering. A short-lived, booking-scoped GUEST token is minted by the same RS256 machinery, carried in its own `__Host-`-prefixed cookie so it can never overwrite a signed-in account session. Services opt in to guest tokens explicitly (default deny); out-of-scope access answers 404, never 403, so booking existence cannot be probed. A live bug where a leftover guest cookie answered for the account that owned a booking was traced to gateway credential precedence and fixed so the account session always wins unless the guest page asks otherwise.

Fare rules — the Premium refund waiver

What & why. Fare families now behave like real ones: Premium is fully refundable and freely changeable; Saver keeps a cancellation fee; the time-tier ladder (full refund ≥ 72 h, 50 % to 24 h, none inside the same-day window) still applies to the others. Reported as a live bug — a Premium passenger being charged a cancellation fee — and fixed at the root.

Engineering. The refund tier is computed once by booking-service (the quoting authority) and rides the CANCELLED event to payment-service, which applies it **per fare line** rather than to a pooled total — so a mixed Premium+Saver booking no longer refunds both at the Saver rate. Recomputing the time-sensitive tier downstream would let two clocks disagree across the seconds between quote and capture; carrying it on the event removes that class of error.

Idempotency — safe retries on money paths

What & why. A retry — a double-click, a lost response, a message redelivery — must never charge twice or refund twice. This closed a defect that was moving real money.

Engineering. The live defect: a redelivered partial-cancellation event passed a status guard that accepted the very state the first refund produced, issuing a **second refund and a second gateway call**. Refunds now carry a cause key unique per payment, enforced by the database; booking and payment creation carry a client idempotency key plus a request fingerprint, so a retry replays the original and a key reused with a different body is refused. Authorize

and capture became idempotent by state — an already-captured payment replays itself instead of reporting a false error on money that already moved.

Mobile-responsive redesign

What & why. The whole app was re-fitted for phones after real-device testing on an iPhone: a measured 40 controls under the 44-pixel tap-target minimum, an over-wide header, and content clipped off the right edge.

Engineering. Fixes were measured, not guessed: a header that demanded 517 px of a 390 px screen collapses to a drawer; a global 44-pixel tap-target floor and a 16-pixel input-font floor (to stop iOS Safari’s focus-zoom) were set once in the design system; pickers became bottom sheets; the booking stepper, trip cards and “My trips” filters were rebuilt to fit.

Flights list pagination

What & why. A latent operational hazard: the list-all flights endpoint loaded the entire ~920,000-row seed network into memory on every call and failed as a misleading 401.

Engineering. Replaced with a paged query (default 50, hard cap 200) issuing LIMIT/OFFSET plus a count. Nothing in the app ever consumed the full list — shopping runs through search and itinerary endpoints — so the fix is pure hardening.

Part 2 — The engineering toolchain

SkyBook is deliberately built the way a funded engineering team would build it. This section names every load-bearing tool, its measured version, and the job it does — logging, monitoring, quality gates, CI/CD, messaging, orchestration and the rest — so the platform’s operational maturity is visible rather than implied.

At a glance

Concern	Tool	Version	Role in SkyBook
Runtime	Spring Boot	3.5.16	Every backend service; Java 21
Runtime	React + Vite + TypeScript	19 / 8 / 6	Single-page front end, type-checked
Messaging	Apache Kafka	3.9.0	The event backbone between services
Database	PostgreSQL	16	One database per service
Migrations	Flyway	core + pg	Versioned, validated schema; no auto-DDL in prod
Metrics	Prometheus	v3.2.1	Scrapes every service’s /actuator/prometheus
Dashboards	Grafana	11.6.0	Fleet dashboards over metrics, logs and traces
Logs	Loki + promtail	3.4.2	Centralised structured (JSON) log aggregation
Tracing	Tempo + OpenTelemetry agent	2.7.1 / 2.14.0	Distributed traces across the Kafka hops
Quality gate	SonarCloud	CI-gated	Coverage, bugs, vulnerabilities, security rating
Image scan	Trivy	CI-gated	Scans every image before it is pushed
Code scan	CodeQL	CI-gated	Static security analysis on the source
CI/CD	GitHub Actions	Managed	Build-once, multi-arch, promote-many pipeline
Containers	Docker + buildx	Compose	Multi-architecture images (amd64 + arm64)
Orchestration	Kubernetes	28-file tree	Full manifest set on a branch, cluster-verify pending
Resilience	resilience4j	spring-boot3	Circuit breakers, bulkheads, retries on outbound calls
Auth	RS256 JWT (jjwt)	0.12.x	Asymmetric tokens; auth-service is sole minter
Frontend tests	Vitest	4.x	Component and behaviour tests

Observability — logs, metrics, traces

SkyBook runs the three pillars, not one. **Metrics**: every service exposes Micrometer/Prometheus metrics on an internal-only management port; Prometheus scrapes them and Grafana renders a fleet dashboard. **Logs**: each service logs structured JSON via Logstash encoders; promtail ships them to Loki, queryable beside the metrics in Grafana. **Traces**: an OpenTelemetry Java agent instruments every service and exports to Tempo, so a single booking can be followed across the gateway, the services, and the Kafka events between them — the reason distributed tracing exists.

The management surface is deliberately isolated: actuator endpoints live on a separate internal port, never on the public business port, so metrics and health are scrapeable inside the network but invisible from the edge.

Quality gates — Sonar, Trivy, CodeQL

Three independent gates guard every change. The **SonarCloud quality gate** blocks a merge on coverage, bugs, vulnerabilities and security rating — it has been driven to a passing state at over 90 % coverage with zero bugs and zero vulnerabilities. **Trivy** scans each container image for known CVEs before it is allowed to be pushed to the registry (scan-before-push). **CodeQL** runs static security analysis over the source. None of the three is advisory; each can stop the pipeline.

CI/CD — build once, promote many

The pipeline (GitHub Actions) builds every image **once** per commit as a multi-architecture manifest — amd64 for the runners, arm64 for the production VM — scans it, and pushes it to the registry tagged by commit SHA. From that moment nothing is rebuilt: the promotion workflow walks the **same digests** up an environment ladder, so what production runs is bit-for-bit what the tests certified.

The ladder: LOCAL → DEV → SIT → TEST/QA → PERF → UAT → STAGING → PROD, with a weekly DR drill. Each rung answers one question — does it boot, do the services agree, does the customer journey hold end to end, is it fast enough (k6 thresholds), does a human accept it, does it run on the real host — and any rung can stop the release. Configuration varies per rung the twelve-factor way; code never branches on environment name.

Messaging — the Kafka event backbone

Services do not call each other to announce facts; they publish events. A confirmed booking emits an event that payment-service, check-in-service and notification-service each consume independently — payment reconciles money, check-in opens, the passenger is emailed a ticket — with no service blocking on another. Consumers are written to be idempotent, and dead-letter topics with bounded retry back every listener, so a transient failure is retried and a poison message is quarantined rather than blocking the partition.

Data — PostgreSQL and Flyway

Each service owns its own PostgreSQL database — no shared schema, no cross-service joins. Schema is owned by **Flyway**: every change is a numbered, immutable migration, applied and validated at start-up; Hibernate is set to validate-only in production, never to generate DDL. The idempotency and fare-rule work in this addendum shipped as new Flyway migrations, so the schema history is a complete, replayable record.

Orchestration — Docker Compose and Kubernetes

The running platform is Docker Compose: multi-architecture images, loopback-bound ports behind a single Caddy TLS front door, internal services unpublished. A **complete 28-file Kubernetes manifest tree** — every service, the datastores, and the full observability stack — exists on a dedicated branch, with the health-probe endpoints (/livez, /readyz) already shipped on the main line as its groundwork. It awaits a cluster to be verified against; the design is finished, not aspirational.

Resilience and security

resilience4j wraps every outbound service call in a circuit breaker, bulkhead and retry, with the thresholds tuned against the Grafana dashboards. Security is asymmetric-signed throughout: **RS256 JWTs** minted only by auth-service and verified locally by every service with a shared public key; per-service machine credentials with least-privilege audiences; ownership enforced at the object level. The public key verifies, the private key — held in one service — signs.

Test evidence

The features in Part 1 are covered by automated tests at every level — unit, integration (Testcontainers against real PostgreSQL and Kafka), an end-to-end certification suite against the promoted images, and front-end component tests. Representative measured backend totals at the time of writing:

Service	Tests	Result
booking-service	490	pass
flight-service	220	pass
auth-service	183+	pass
payment-service	126	pass
checkin-service	157	pass
api-gateway	20	pass
skybook-security (shared)	83	pass
Front end (Vitest)	76	pass

Numbers are point-in-time and grow with each increment; the pipeline's gates, not this table, are the authority on any given commit.